

Contexts that move

A theory of references, debts and repairs for multipass randomization
with planetary context shifts

Investigation for exfret's property randomizer (`propertyrandomizer`)

26 September 2026

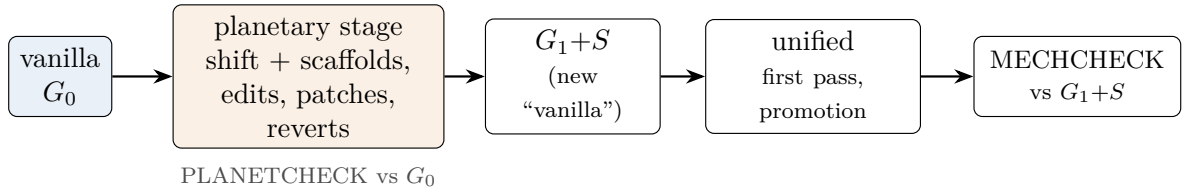
Summary

1. **Unified never needed vanilla.** It needs a *reference*: any world plus a fallback assignment that reaches the goals, with a sort of it. Promotion's two guarantees hold for every reference: soundness, and a fallback that is always available (Theorems 3.1–3.2, Corollary 3.4). Multipass is a *relay of references*: each pass receives one and must hand one on.
2. **A context shift is the one pass that changes the goals.** Feature goals follow the feature, identity goals stay (with relaxed isolatability), recipe goals stay. The planetary check's three rules are exactly this transport. The dilemma (Proposition 4.5): vanilla is a reference but cannot see moved features, and the shifted world sees them but is not a reference for the transported goals.
3. **Three ways to restore a reference.** *Scaffolded* (today: fix the world before unified), *repaired* (change the fallback, which is visible only where unified falls back to it), and *superposed* (old and new features at once, carrying debts that are settled after unified along a ladder: paid, repair, duplicate, addition, revert). Every planetary feature enters the logic through edges into $\vee$ nodes, so the superposition is *always* a reference (Lemma 5.4). In the measurements it lost nothing in all 24 raw-shift runs.
4. **Contradictions are few when counted by cause.** A raw ocean swap fails 151 goal pebbles on every seed. They trace back to the 4 lost oceans and are fixed by 10 irreducible slot repairs. The combined ocean and resource swap fails 76–166 goal pebbles, which come from 7–9 causes and are fixed by 11–15 repairs (Table 3, Figure 9).
5. **Most ocean repairs lie outside unified's current move space.** On every seed, 7 of the 10 ocean repairs change a slot unified never randomizes: water or lava ingredients, or a blacklisted recipe. Restricted to unified's own move space, nearly every contradiction is still fixable, but only by *leaf* repairs in consumers such as rocket parts and silos. That takes 26 slots instead of 10 for the ocean swap.
6. **Some contradictions are structural.** When Vulcanus loses calcite (seeds 3, 5 and 7), eight goal pebbles cannot be fixed by *any* ingredient change. `calcite-processing` is researched by mining calcite, and `foundry` requires it. Only an extra calcite patch (the addition rung, which the resource stage already implements) or a different trigger fixes them. One staged sort finds such cases (Proposition 6.2).
7. **Recommendation.** First, have the planetary stage produce a *repaired reference*: staged-sort repair selection, in-place edits unless a context conflict forces a duplicate. That needs no change to unified. Then add repair-only moves (pins) for blacklisted slots. Finally, put a superposed reference with debt-aware promotion behind a flag, starting with resources, whose addition rung already exists. Five decisions are the user's to make (Section 7).

Contents

1	Introduction	3
1.1	Two pipelines that disagree about contexts	3
1.2	The question	3
1.3	What this report does	4
2	The logic as a Horn program over pebbles	4
2.1	Graph, contexts, pebbles	4
2.2	Rules	4
2.3	Monotonicity, and why home contexts matter	5
2.4	Randomization as an assignment	5
3	What promotion needs: a reference, not vanilla	6
3.1	Promotion as an invariant	6
3.2	References and the relay between passes	7
4	Context shifts	8
4.1	Feature edges	8
4.2	Goal transport	9
4.3	The dilemma	9
4.4	Chunks: why shifts are not monotone	10
5	Three ways to build a reference for shifted goals	10
5.1	Scaffolded reference (today)	11
5.2	Repaired reference: edit the fallback, not the world	11
5.3	Superposed reference: carry the debt, settle later	12
5.4	The settlement ladder	13
5.5	Comparison	14
6	Side investigation: fixing a small number of contradictions	14
6.1	Setup of the measurements	15
6.2	Contradictions are counted in the wrong unit	15
6.3	Repairs as relaxations	15
6.4	Staged sorts: witnesses that repair only where needed	16
6.5	Where the repairs are	17
6.6	Context conflicts: when a repair must be a duplicate	20
6.7	What this means for “unified fixes the contradictions”	21
7	Putting it together	22
7.1	A pipeline with shifts inside the relay	22
7.2	Context-shifting first pass	22
7.3	Recommendations, cheapest first	23
7.4	Decisions that are the user’s	23
A	Experiment details	24
B	New terms	25

Today



Proposed

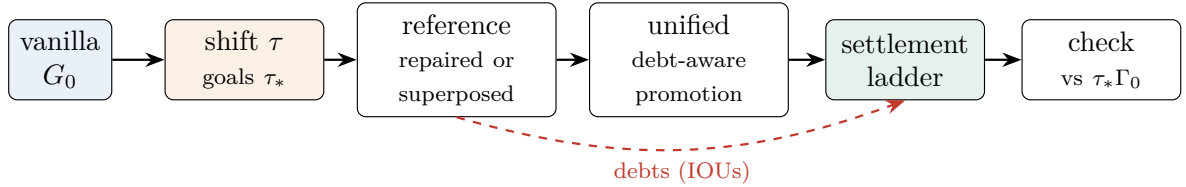


Figure 1: Top: the current order. The planetary stage has to produce a world that is completable on its own, and unified treats it as vanilla. Bottom: the pipeline this report argues for. The shift transports the goals; unified runs against a *reference* that is repaired or superposed; whatever the reference owes (its debts) is settled after unified, and the result is checked against the transported vanilla goals.

1 Introduction

1.1 Two pipelines that disagree about contexts

Unified randomization keeps a game completable by reasoning relative to a fixed *reference*: a random contextual sort of a world that is already known to be completable. Every choice it makes (a new base for a head, a new ingredient list for a recipe) is accepted only if every promised pebble can still be backed by strictly earlier pebbles of that sort (`randomizations/graph/unified/skeleton/promotion.lua`). If nothing better is found, the reference’s own choice (the vanilla base, the vanilla ingredients) is always available as a fallback. Mechanic contexts are hard constraints: rooms and automatability are always kept, isolatability where declared (`skeleton/protection.lua`).

Planetary randomization moves *features* between planets: ocean tiles and their fluid, resource placements, lightning, freezing (`randomizations/planetary/*.lua`). This changes, on purpose, which contexts mechanics have: lava is pumped on another planet, Vulcanus pumps something else. So planetary randomization cannot keep “every mechanic keeps all its contexts”. It has its own rules instead (`randomizations/planetary/check.lua`, `protection.planetary_kept_context`): recipes stay reachable somewhere, planet-locked recipes keep their exact contexts, mechanics keep rooms and automatability, isolatability only for rocket building and electricity, and moved features follow their feature.

The two are joined today by running planetary randomization *first*, as a separate stage in `data-final-fixes.lua`. It must leave behind a world that satisfies its rules by itself, using scaffold recipes (variants of recipes that used a planet’s old fluid), in-place recipe edits, extra resource patches and climate reverts. Unified then takes that world as “vanilla”: `old_data_raw`, the initial sorts and the final MECHCHECK are all taken after the planetary stage (Figure 1, top).

1.2 The question

The user asked for a theoretical basis for multipass systems in which a pass may shift contexts the way planetary randomization does, and for a side investigation into letting later passes fix “a

small number of contradictions” that such a shift creates. The motivation is integration: unified should be able to fix some of planetary’s contradictions itself, instead of the planetary stage settling everything up front with scaffolds. Two facts stand in the way:

1. planetary randomization changes contexts, while unified’s hard constraint is that mechanics keep theirs;
2. unified relies on a completable version of the game, both to rank pebbles and as the fallback that makes it always able to finish.

1.3 What this report does

- Section 2 states the logic as a Horn program over pebbles, so that “reachable”, “sort” and “backing” have exact meanings, and shows why the order-independent discovery rule (home contexts) matters for everything that follows.
- Section 3 isolates what promotion actually needs: a *reference*, meaning a completable world with a proof order. Vanilla is one reference among many. Multipass is a relay of references.
- Section 4 defines context shifts, goal transport and contradictions. It proves the dilemma behind the user’s question: vanilla is a reference but cannot see moved features, and the shifted world sees them but is not a reference.
- Section 5 gives three ways out: *scaffolded*, *repaired* and *superposed* references. The last one carries explicit debts that are settled after unified, along a ladder of increasingly visible fixes.
- Section 6 is the side investigation. It measures contradictions on real Space Age seeds (goals, causes, repairs) and gives a staged sort that finds small repair sets. It also finds where unified’s current move space does and does not cover the repairs.
- Section 7 assembles a concrete pipeline and lists the decisions that are the user’s to make.

Game facts used here were checked against the installed 2.1 game data and prototype documentation (/Applications/factorio.app/Contents/data, doc-html/prototype-api.json, version 2.1.20); where one is stated, its source is given next to it.

2 The logic as a Horn program over pebbles

This section fixes notation. Nothing in it is new to the code; it states exactly what `lib/graph/context-sort.lua` computes, so the later arguments can be precise.

2.1 Graph, contexts, pebbles

The logic graph $G = (N, E)$ has, for every node n , an operator $\text{op}(n) \in \{\wedge, \vee\}$ and a context type $\kappa(n)$: a *transmitter* (`type_info[...].context=nil`), a *forgetter* (`true`, e.g. technologies) or an *emitter* of one room (a string, the room nodes) (`lib/logic/builder.lua`, comment above `add_node`). An edge e may carry abilities $\alpha(e)$, which gain or lose isolatability (i) or automatability (a) for whatever passes through it.

With complex contexts, a context is a room together with an ability string, $\mathcal{C} = \text{Rooms} \times \{0, 1\}^{\{i, a\}}$, written like `planet:vulcanus|10`. Home contexts add one context per room and home set, riding on the room’s weakest context (`...|00@home1`). Contexts of one room are ordered by their abilities: $c \preceq c'$ when c asks for a subset of what c' asks for. Anything reachable in c' is reachable in every $c \preceq c'$; this is why promises are downward closed (`weaker_contexts` in `promotion.lua`).

A *pebble* is a pair $\langle n, c \rangle \in N \times \mathcal{C}$.

2.2 Rules

The sort’s transmission rules are all of one shape: a pebble follows from finitely many other pebbles. Writing $e = (m, n)$ *delivers* c when some reached $\langle m, c' \rangle$ has $c \in \alpha_e(c')$ (`edge_transmit`):

- an $\wedge$ node *fires* in c when every in-edge delivers c ; an $\vee$ node when some in-edge does; an $\wedge$ node without in-edges (a source) fires in every context;
- a node that fires in c sends out $\text{out}_n(c)$: $\{c\}$ for a transmitter; the same abilities in every room for a forgetter, except that isolatable contexts stay in their room; every ability string of its room for an emitter (`node_transmit`);
- home contexts go through transmitters unchanged, are emitted by a room only for home sets containing it, and are spread by forgetters to every room (`home_transmit`);
- the discovery rule: once a discoverer of room r is reached, a technology with a home context of r 's home set gets r 's isolatable contexts (`discover_home_rooms`, `add_home_tech`).

Each rule has the form “ p holds if $q_1, \dots, q_k$ hold”, a definite Horn clause over pebbles. Let $\Pi(G)$ be the resulting finite propositional program.

Definition 2.1 (Reach, sort, witness). $\mathcal{R}(G)$ is the least model of $\Pi(G)$: the pebbles derivable from the sources. A *sort* (proof order) is an enumeration $\pi = (p_1, p_2, \dots)$ of $\mathcal{R}(G)$ in which every p_i has a rule instance whose body lies in $\{p_1, \dots, p_{i-1}\}$. We write $\text{rk}_\pi(p)$ for p 's index. A *witness* of p is the backward closure of p under chosen rule instances; `top.path` builds one with the earliest-provider rule.

`top.sort` computes $\mathcal{R}(G)$ together with one sort, breaking ties at random, so every call returns a different sort of the same least model.

Lemma 2.2 (Justification). *For a set S of pebbles, $S \subseteq \mathcal{R}(G)$ if and only if there is a rank function $\rho : S \rightarrow \mathbb{N}$ such that every $p \in S$ has a rule instance of $\Pi(G)$ whose body B satisfies $B \subseteq S$ and $\rho(q) < \rho(p)$ for all $q \in B$.*

Proof. If ρ exists, induction on ρ shows each $p \in S$ is derivable. Conversely, the ranks of any sort work. $\square$

Lemma 2.2 is the glossary's *well-founded* condition, and it is the whole correctness argument of promotion (Section 3).

2.3 Monotonicity, and why home contexts matter

Lemma 2.3 (Monotonicity). *If G' comes from G by adding in-edges to $\vee$ nodes, or by removing in-edges of $\wedge$ nodes, then $\mathcal{R}(G) \subseteq \mathcal{R}(G')$.*

Proof. Adding an $\vee$ in-edge adds rules to Π ; removing an $\wedge$ in-edge shrinks rule bodies (removing the last one makes the node a source, which fires everywhere). Least models of definite programs only grow under both. $\square$

Remark 2.4 (The old discovery rule is not a rule). Without home contexts, `discover_space_locations` gives isolatable contexts of a room to the technologies reached *before* its discoverer *in this sort*. That is not a Horn clause: which pebbles exist depends on π , so $\mathcal{R}$ is not a function of G . Neither lemma holds. Adding an edge that lets a discoverer come earlier can *remove* isolatable contexts, because fewer technologies come before it. Home contexts turn the rule into a clause (technology home pebble + discoverer pebble $\Rightarrow$ isolatable pebble), which is why they fixed the order-dependence failures in promotion and monotone matching. Everything from Section 4 on uses Lemma 2.3, so it assumes home contexts are on, as they are in unified, the planetary check and MECHCHECK today.

2.4 Randomization as an assignment

A randomized edge $m \rightarrow n$ is subdivided into $m \rightarrow b \rightarrow h \rightarrow n$ with a base b ($\wedge$) and a head h ($\vee$), and the $b \rightarrow h$ connection is cut (`gutils.subdivide_base_head`; “Cut base-head connections” in `execute-new.lua`). An *assignment* σ maps heads to bases; $G[\sigma]$ is the cut graph

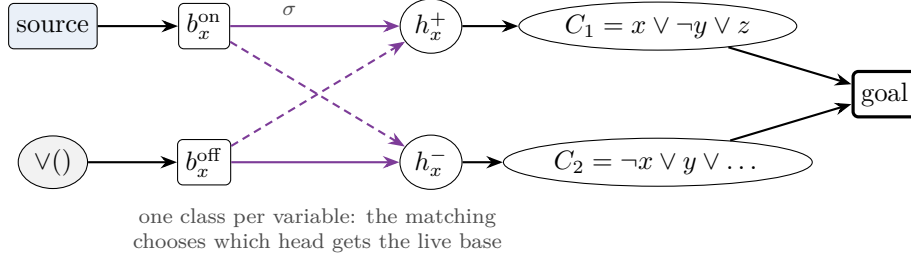


Figure 2: The gadget behind Proposition 2.5. Solid purple: one of the two perfect matchings of a variable’s class; dashed purple: the other.

with every $\sigma(h) \rightarrow h$ connected. Handlers constrain σ differently: first pass wants a perfect matching of item identities to slots, recipe ingredients may reuse bases. The reference assignment σ_{ref} connects every head to its old base.

The goals Γ are the pebbles a result must reach:

- Γ^{mech} : mechanic pebbles, reduced to the part `protection.kept_part` says must be kept;
- Γ^{rec} : for every recipe r reachable in the reference, *some* pebble of r .

The second kind is disjunctive, which is why promotion picks an anchor context late (`required_contexts`).

Proposition 2.5 (Randomizing with guarantees is NP-complete). *Deciding whether a matching σ exists with $\Gamma \subseteq \mathcal{R}(G[\sigma])$ is NP-complete, even with a single context and no abilities.*

Proof. Membership: guess σ , compute $\mathcal{R}$. Hardness from 3-SAT (Figure 2): for each variable x make two heads h_x^+ , h_x^- and two bases, b_x^{on} (fed by a source) and b_x^{off} (fed by an $\vee$ node with no inputs, so never reached), matched within their class. The literal node for x is h_x^+ , for $\neg x$ it is h_x^- . Each clause is an $\vee$ of its literal nodes, and the goal is an $\wedge$ of the clauses. A perfect matching puts b_x^{on} on exactly one of $h_x^\pm$, which is a truth assignment, and the goal is reached exactly when it satisfies every clause. $\square$

So an unrestricted “choose everything, then check” randomizer is solving an NP-complete problem each time. Monotone matching and promotion avoid this by fixing a sort first, which turns each choice into a local test against ranks. Section 3 makes that trade explicit, because it is also exactly what a context shift breaks.

3 What promotion needs: a reference, not vanilla

3.1 Promotion as an invariant

Promotion (`skeleton/promotion.lua`) works on a graph whose unresolved heads are connected to their reference bases (the “hybrid graph” built and sorted at lines 54–81) and takes one random sort π of it. It keeps a set Q of *promised* pebbles and an assignment σ_{cur} : resolved heads have their chosen base, unresolved heads their reference base. Its operations are:

establish(p) search for a backing of p in $G[\sigma_{\text{cur}}]$ from pebbles of strictly lower π -rank that are themselves established (promised pebbles count as established);

commit(p) add p , its whole backing and every weaker context of each (downward closure) to Q ;

resolve(h, b, C) accept base b for head h if, in each required context $c \in C$, b has an established pebble ranked before the head’s (with edge abilities: a pebble whose context the connecting edge turns into c , as `base_backing` checks); then set $\sigma_{\text{cur}}(h) = b$ and re-commit the dependent.

The invariant is:

$$\text{every } q \in Q \text{ has a backing in } G[\sigma_{\text{cur}}] \text{ made of pebbles of } Q \text{ with lower } \pi\text{-rank.} \quad (\text{W})$$

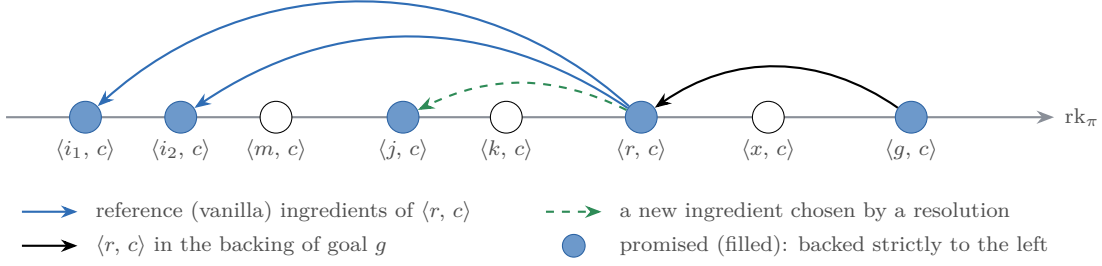


Figure 3: Promotion’s invariant on one sort. The recipe pebble $\langle r, c \rangle$ was promised as part of goal g ’s backing, so its reference (vanilla) ingredient pebbles $\langle i_1, c \rangle, \langle i_2, c \rangle$ are promised too, which is why they stay available as a fallback (Theorem 3.2). A resolution may replace them by any established pebble to the left, such as $\langle j, c \rangle$.

Theorem 3.1 (Soundness). *If (W) holds when randomization ends, then $Q \subseteq \mathcal{R}(G[\sigma_{\text{cur}}])$. In particular every promised mechanic pebble is reachable in the final game, provided reflection builds the game the model describes.*

Proof. Lemma 2.2 with $\rho = \text{rk}_\pi$. □

The proviso is not idle: when reflection and the model disagree (the item reflection mismatch found earlier), the theorem is about a different game. That is why UNIFIEDCHECK and MECHCHECK rebuild logic from the final `data.raw` (`skeleton/check.lua`).

Theorem 3.2 (The fallback is always available). *Let h be an unresolved head whose dependent d is promised in every required context $c \in C$. Then $b = \sigma_{\text{ref}}(h)$ passes the test of $\text{resolve}(h, b, C)$.*

Proof. Heads only ever feed $\wedge$ nodes: claiming subdivides in-edges of $\wedge$ nodes only, and `gutils.make_orands` first turns every $\vee$ in-edge into an $\wedge$ “orand” node (`execute-new.lua`, CLAIMING). So every backing of $\langle d, c \rangle$ uses $\langle h, c \rangle$. While h is unresolved, its only in-edge in $G[\sigma_{\text{cur}}]$ comes from $\sigma_{\text{ref}}(h)$, so that backing goes through a pebble of $\sigma_{\text{ref}}(h)$ ranked before $\langle h, c \rangle$. Committing $\langle d, c \rangle$ committed the backing, so the pebble is in Q and is established. Commits never remove promises (only `try_rewires` un-promises, and it restores them on failure), so this still holds when h is resolved. □

This is the glossary’s *fallback bundle*: “when the sort is a random sort of the vanilla graph, the recipe’s vanilla ingredients are always a fallback bundle”. Recipes that are not yet promised anywhere get an anchor context only when they are resolved, which keeps the choice of context flexible. It also means earlier choices can cut such a recipe off first; promotion then fails the attempt instead of losing the recipe silently. `promise_single_context_recipes` closes the gap for recipes with only one room.

3.2 References and the relay between passes

Nothing in either theorem mentions vanilla. They use two facts: π is a sort of $G[\sigma_{\text{ref}}]$, and the goals are reachable there.

Definition 3.3 (Reference). A *reference* is a tuple $\mathcal{R} = (G, \sigma_{\text{ref}}, \pi, \Gamma)$ where $\Gamma \subseteq \mathcal{R}(G[\sigma_{\text{ref}}])$ and π is a sort of $G[\sigma_{\text{ref}}]$.

Corollary 3.4 (Promotion only needs a reference). *Theorems 3.1 and 3.2 hold for any reference. The world may be the vanilla game, a game changed by earlier stages, or a model graph such as first pass’s split graph, and the fallback assignment may be any assignment whose graph reaches the goals.*

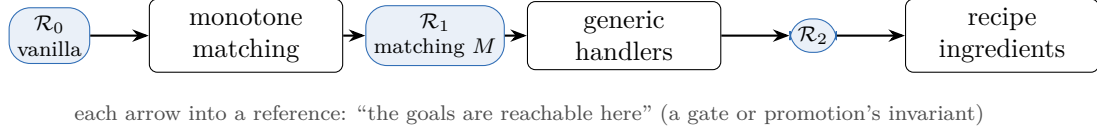


Figure 4: Multipass as a relay of references. A pass may use any sort of the reference it receives, and it must hand on a world in which the goals are still reachable. A context shift is a pass that cannot do this (Section 4).

This is already how the pipeline works. After planetary randomization, `old_data_raw` and the initial sorts are taken from the changed world, so unified’s reference is not vanilla. With first pass on, promotion “must reason over first pass’s split graph” (`execute-new.lua`), whose reference assignment is the matching monotone matching chose. Monotone matching itself runs in rounds, and each round’s matching is the reference of the next (“needs only grow and the current matching always passes”, `monotone-matching.lua`).

Definition 3.5 (Monotone pass). A pass given a reference $(G, \sigma_{\text{ref}}, \pi, \Gamma)$ is *monotone* if its output assignment σ' gives every goal a π -monotone backing in $G[\sigma']$.

A monotone pass hands on a new reference $(G, \sigma', \pi', \Gamma)$, where π' is any sort of $G[\sigma']$. So **multipass is a relay of references**: each pass needs a completable world with a sort from the pass before it, and must hand one on (Figure 4). Vanilla is only where the relay starts. The item first pass’s admissibility rule (a trav moves only to a slot that supplies its needed contexts earlier in the current sort) is exactly the condition that makes it monotone. Its gate checks the handover.

4 Context shifts

4.1 Feature edges

A planetary feature enters the logic as an edge into a node that says the feature is present or usable. For resources, lightning and freezing the edge comes from a room-level node. For oceans it comes from the tile’s pump node, because the ocean swap keeps the tile and changes its fluid. Table 1 lists the four features planetary randomization moves today. Every one of these edges ends in an $\vee$ node. That is the property the rest of the report leans on.

Feature	Feature edge in the logic	Target op
oceans	<code>tile-fluid:t → fluid-create-offshore-temperature:f</code> . The ocean swap reskins slot tiles, so the tile keeps its room and the edge’s target fluid changes (<code>concrete.lua</code> , <code>fluid-create-offshore-temperature</code> ; <code>oceans.lua</code>)	$\vee$
resources	<code>room-autoplace:P → entity:R</code> for resource placements (<code>add_autoplace_edges</code> in <code>concrete.lua</code> ; <code>resources.lua</code> moves the planet’s autoplace settings)	$\vee$
lightning	<code>room:P → energy-source-electric-production-lightning-existence</code> , gaining isolatability, for planets with <code>lightning_properties</code> (<code>abstract.lua</code>)	$\vee$
freezing	<code>room:P → warmth</code> for planets <i>without</i> <code>entities_require_heating</code> ; freezing is the absence of this edge (<code>abstract.lua</code>)	$\vee$

Table 1: Planetary features as feature edges. In vanilla Space Age, Fulgora is the planet with `lightning_properties` (`data/space-age/prototypes/planet/planet.lua:320`) and Aquilo the one with `entities_require_heating=true` (same file, line 680). For oceans and resources the experiment in Section 6 also checks this mechanically: every edge that exists only in the unshifted graph ends in an $\vee$ node.

Definition 4.1 (Shift). Let $E_F \subseteq E$ be the feature edges. A *shift* is a pass that replaces E_F by another set E'_F of feature edges over the same nodes and leaves everything else alone: $G_1 = (G_0 \setminus E_F) \cup E'_F$. Ocean and resource swaps are permutations of features among rooms. Climate changes add or remove single edges.

A shift is a handler whose heads are feature edges. What sets it apart from recipe or item randomization is not the kind of edge. It is that the *goals are defined relative to it*: after the swap nobody wants lava on Vulcanus.

4.2 Goal transport

Definition 4.2 (Transported goals). Classify each goal of Γ_0 by what its node declares:

- *feature goals*: mechanic nodes built with a moved `planetary_feature`. They *follow the feature*. The current rules drop them (`planetary_kept_context` returns `nil`); full transport would require $\langle n, \phi(c) \rangle$ instead, where ϕ moves c 's room to wherever the feature went;
- *identity goals*: every other mechanic pebble. It is kept with its room and automatability, and its isolatability only if the node is built with `keep_planetary_isolatability` (rocket building and electricity);
- *recipe goals*: every recipe stays reachable somewhere (rule 1), and recipes locked to one planet by surface conditions keep their exact contexts (rule 2).

Write $\Gamma_1 = \tau_*\Gamma_0$. These are exactly the three rules of `check.required_failures` in `randomizations/planetary/check.lua`.

The classification is declared on the nodes themselves, through the flags `planetary_feature` and `keep_planetary_isolatability`, which fits the project's "declare on nodes, not names" rule. Transport is a different goal specification, not a weaker one. Identity goals get weaker (less isolatability), while feature goals move.

Remark 4.3 (Home sets are part of the goal specification). The discovery rule grants a technology the isolatable contexts of room r when it can be had using only r 's *home set*: the rooms that r 's discoverers cannot be reached without. Home sets are computed once from vanilla and kept fixed (`logic.home_sets`, and `check.home_sets` in the planetary check). A shift could in principle change which rooms a discovery needs, for example if a discovery technology's science came to depend on a moved resource. For any fixed set the rule stays a Horn clause, so every lemma here still holds; only its meaning is at stake. A *smaller* set grants less, so the conservative choice after a shift is the intersection of the vanilla home sets and those recomputed on the shifted world. For the current shifts this matters only if a discovery technology's requirements come to depend on a moved feature, and checking that costs one extra `top.home_sets` pass on the shifted graph.

Definition 4.4 (Contradictions). The *contradictions* of a shift are $D = \Gamma_1 \setminus \mathcal{R}(G_1[\sigma_{\text{ref}}])$: transported goals that the shifted world does not reach with the reference assignment.

4.3 The dilemma

Proposition 4.5 (Neither world is a usable reference).

- (a) *With the vanilla reference $(G_0, \sigma_{\text{ref}}, \pi_0, \cdot)$, promotion can only establish pebbles of $\mathcal{R}(G_0[\sigma_{\text{ref}}])$. Pebbles that exist only through a moved feature edge, such as lava at its new planet, are never available, and a pass that removes an old feature edge is accepted only if every promise still has a backing through vanilla pebbles.*
- (b) *The shifted world $(G_1, \sigma_{\text{ref}}, \pi_1, \Gamma_1)$ is a reference exactly when $D = \emptyset$.*

Proof. (a) Promotion ranks pebbles by π_0 , a sort of $\mathcal{R}(G_0[\sigma_{\text{ref}}])$, and `establish` only looks at ranked pebbles. (b) is Definition 3.3. $\square$

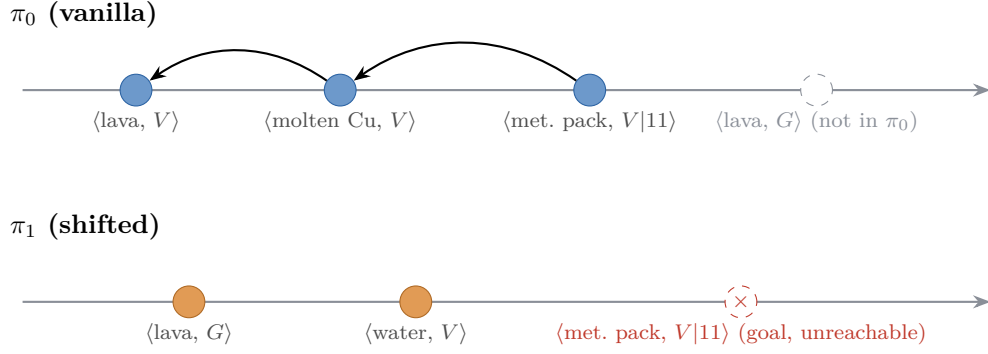


Figure 5: The dilemma of Proposition 4.5, on the Vulcanus science goal. In the vanilla sort, the goal has its backing but the moved lava does not exist. In the shifted sort, the new features exist but the goal does not, so promotion cannot promise it.

Part (a) is what a planetary handler inside today’s unified would look like. It could *remove* feature edges through `try_rewires` (the check that every promised pebble keeps a backing), but it could never *use* what it adds. Every identity goal that relied on lava would have to be re-routed through vanilla pebbles of Vulcanus, and in practice there are none: that is why lava is on Vulcanus. Part (b) is today’s situation, where the planetary stage must bring D to zero before unified starts. Figure 5 shows both.

4.4 Chunks: why shifts are not monotone

Fix a witness W of an identity goal in G_0 that uses a feature pebble p_f which the shift removes. The *chunk* of W at p_f is the set of pebbles of W that p_f dominates, meaning every path in W from them down to the sources passes through p_f . After the shift the chunk has no root. There are only two ways to repair W (Figure 6):

- a **root repair** gives the chunk’s lowest consumers something else that is available in the same context (molten copper from the new local fluid instead of lava). This is what planetary scaffolds do;
- a **leaf repair** re-derives the pebbles above the chunk without it (the goal recipe takes an ingredient that does not need molten copper). This is what unrestricted ingredient randomization would do.

For a feature goal the chunk moves with the feature instead. Its pebbles above p_f that also need other branches (the calcite next to the lava) now need those branches in the new room. These are the chunk’s *boundary debts*.

This explains two earlier observations. First, “a chunk move always needs a new witness”: monotone matching only ever keeps witnesses, and a shift destroys them by definition. Second, random non-monotone moves were catastrophic in the tempering experiment. Each such move leaves a chunk without a root, and nothing was keeping track of the debt.

5 Three ways to build a reference for shifted goals

By Corollary 3.4, integrating a shift with unified comes down to one question: which completable world, with which fallback assignment, do we hand to unified for the goals Γ_1 ? There are three answers. They differ in when the contradictions are settled and in what the settlement leaves in the game.

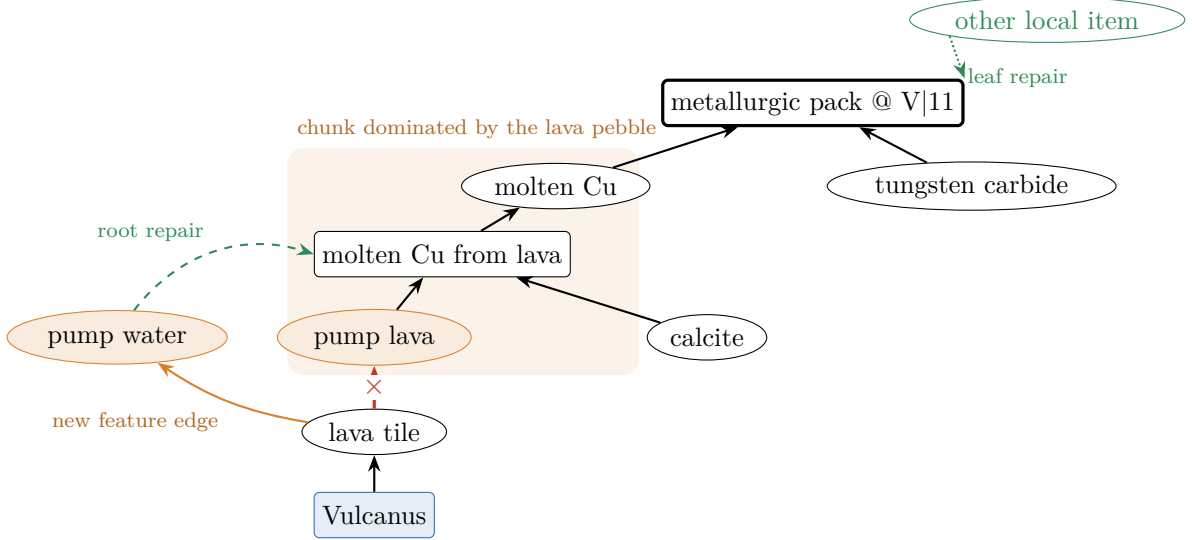


Figure 6: A chunk. The ocean swap turns the Vulcanus lava tile’s pump output into water, so the orange chunk loses its root. A root repair feeds the chunk’s first consumer from the new local fluid. A leaf repair routes the goal around the chunk. Seed 1’s measured repairs on Vulcanus are root repairs of exactly this kind (Section 6). Recipe facts: `molten-copper-from-lava` takes lava and calcite, and `metallurgic-science-pack` takes tungsten carbide, tungsten plate and molten copper and needs pressure 4000, which only Vulcanus has (`data/space-age/prototypes/recipe.lua:776-797, :1418-1434; planet.lua:47`).

5.1 Scaffolded reference (today)

Add prototypes S (variant recipes, conversions, extra patches) and data edits until $\Gamma_1 \subseteq \mathcal{R}(G_1 + S)$, then hand $(G_1 + S, \sigma_{\text{ref}}, \pi, \Gamma_1)$ to unified. This is the current planetary stage, and it is a reference by construction, since PLANETCHECK verifies it. Its costs follow from the theory:

- S is chosen before unified has made any choice, so it cannot use anything unified does;
- duplicates in S are content. The planned “X (Planet)” variants stay in the game whether or not the final backings use them;
- removing unneeded scaffolds after unified did not work (the deleted `remove_redundant`), and Theorem 3.2 says why. Once S is in the reference, promotion’s backings use scaffold pebbles whenever they are earliest, and nothing prefers backings without them.

5.2 Repaired reference: edit the fallback, not the world

Definition 5.1 (Repair). A *repair* is a change Δ of the reference assignment inside some later pass’s slot space, such as a recipe ingredient slot getting a different base. It is chosen so that $\Gamma_1 \subseteq \mathcal{R}(G_1[\sigma_{\text{ref}} \oplus \Delta])$.

Theorem 5.2. *If Δ is a repair, then $(G_1, \sigma_{\text{ref}} \oplus \Delta, \pi_1, \Gamma_1)$ is a reference, so promotion is sound and always able to fall back when run against it. For every repaired slot, the fallback is the repaired base.*

Proof. Definition 3.3 and Corollary 3.4. □

The point of repairs is what they leave behind. **A repair is not content:** if unified randomizes the slot, the repair is gone; it shows only where unified falls back, and a fallen-back recipe looks like any other randomized recipe. A scaffold duplicate stays either way. The two kinds of edit the planetary stage already makes are repairs in this sense: an in-place edit of a recipe that only its planet used (`scaffolds.lua`), and a planet-specific recipe following a resource

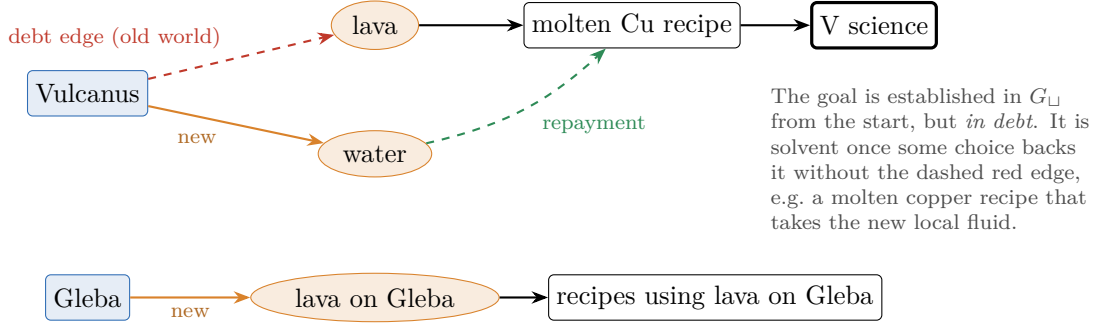


Figure 7: The superposed reference. Every planet keeps its old feature edges as debt edges (dashed red) next to its new ones. Everything that was reachable in either world is reachable here, so promotion can start. Randomization choices that back the goal through the new features *repay* the debt.

swap (`resources.edits`). The theory says to make *every* fix a repair when possible, and to keep duplicates only where a single recipe cannot serve all its contexts (Section 6, context conflicts).

Two practical limits:

- a repaired slot that unified cannot randomize (a blacklisted ingredient such as water or lava, or a blacklisted recipe) keeps the repaired base for good, which is how today’s scaffold edits behave;
- Δ has to be found. That is the “fix a small number of contradictions” problem, treated in Section 6.

5.3 Superposed reference: carry the debt, settle later

Both constructions so far settle every contradiction before unified runs. The third one lets unified run first.

Definition 5.3 (Superposition). $G_{\sqcup} = G_0 \cup G_1$ has every edge of either world. Its *debt edges* $E_{\text{debt}} = E(G_0) \setminus E(G_1)$ are the feature edges that the shift removed.

Lemma 5.4 (The superposition is always a reference). *If every debt edge ends in an $\vee$ node, then for every assignment σ ,*

$$\mathcal{R}(G_0[\sigma]) \cup \mathcal{R}(G_1[\sigma]) \subseteq \mathcal{R}(G_{\sqcup}[\sigma]).$$

In particular $(G_{\sqcup}, \sigma_{\text{ref}}, \pi_{\sqcup}, \Gamma_0 \cup \Gamma_1)$ is a reference, and so is $(G_{\sqcup}, \sigma_{\text{ref}}, \pi_{\sqcup}, \Gamma_1)$ whenever every identity and recipe goal of Γ_1 was reachable in vanilla.

Proof. $G_{\sqcup}$ is G_1 plus $\vee$ in-edges, and G_0 plus $\vee$ in-edges (the new feature edges also end in $\vee$ nodes, Table 1). Apply Lemma 2.3 twice. $\square$

The superposition is the world where every planet has both its old and its new features. Pebbles in it can be *solvent* or *in debt*:

Definition 5.5 (Solvency). A pebble is *solvent* under σ if it has a backing in $G_{\sqcup}[\sigma]$ that uses no debt edge, i.e. a backing in $G_1[\sigma]$. Otherwise it is *in debt*, and its backings need the old world.

Debt-carrying promotion. Run promotion on $G_{\sqcup}$ and track solvency next to establishment:

1. **establish** works as today, in $G_{\sqcup}[\sigma_{\text{cur}}]$, so (W) and Theorems 3.1–3.2 hold unchanged;
2. a second mode, **establish_solvent**, skips debt edges. This is one extra flag in **compute_support**, plus a separate memo;

3. the debt $\Phi(\sigma_{\text{cur}})$ is the number of goals without a solvent backing. To keep it from rising, a resolution may not make a solvent promised pebble insolvent: if $\langle r, c \rangle$ was solvent, its new bases must be solvent in c as well. That is a local check. It suffices because every solvent backing that went through $\langle r, c \rangle$ stays solvent, and it keeps the fallback, because the reference choice leaves the graph as it was;
4. where it matters, the choice is steered: for slots on a goal's repair plan (Section 6), candidates that are solvent in the goal's contexts come first.

In pseudo-code, the changes to `promotion.lua` are small:

```

establish(p, mode)          -- mode: "any" (today) or "solvent"
  if mode == any and p is promised: return true
  for each way to back p with earlier pebbles in G_sup[sigma_cur]:
    if mode == solvent and it uses a debt edge: skip it
    if every pebble q it uses has establish(q, mode): return true
  return false

resolve(r, bases, C)        -- as today, plus the no-new-insolvency rule:
  for each c in C where establish(<r,c>, solvent) held before:
    require establish(<b,c>, solvent) for every new base b

settle()                    -- after every pass
  S <- sort G1[sigma_final], then continue it with the debt gate open
  E* <- debt edges on S's witness of the goals
  for each e in E*: take the first rung of the ladder that works

```

When all passes are done, the goals may still owe something. Find out what with a *solvency-first witness*: sort $G_1[\sigma_{\text{final}}]$ first, then continue the same sort after opening the debt edges (the staged sort of Section 6). The debt edges on that witness, E_{debt}^* , are all that is still owed.

Theorem 5.6 (Settlement). $\Gamma_1 \subseteq \mathcal{R}(G_1[\sigma_{\text{final}}] \cup E_{\text{debt}}^*)$. If $E_{\text{debt}}^* = \emptyset$, the final game is the pure shift, with nothing added.

Proof. The solvency-first witness is a derivation in $G_{\sqcup}[\sigma_{\text{final}}]$ that uses only edges of $G_1[\sigma_{\text{final}}]$ and E_{debt}^* . Apply Lemma 2.2. $\square$

So the question “can unified fix planetary’s contradictions?” becomes “how much of the debt does unified pay?”. Whatever it leaves unpaid, E_{debt}^* , is settled explicitly, in the order below.

5.4 The settlement ladder

Each debt that is still owed at the end climbs the ladder of Figure 8 until one rung settles it. The rungs are ordered by how much they show in the game:

1. **paid**: a randomization choice already backs the goal without the debt. Nothing to do;
2. **repair** (edit the reference): if the debt is settled before unified, a fallback edit (Theorem 5.2). During unified, the same thing is a *pin* on the slot, like promotion’s recycling pins;
3. **planned duplicate**: a planet variant “X (Planet)” for a recipe that must work differently in two rooms (a context conflict, Section 6);
4. **addition**: materialize the debt edge itself (an extra resource patch, a planet keeping its lightning). For oceans this rung is closed by the user’s decision that oceans move as tiles and their fluid changes with them;
5. **revert**: retry the attempt without the shift, which is the true-vanilla fallback the user asked for. It is always available, and it leaves the seed exactly as it would be without the shift.

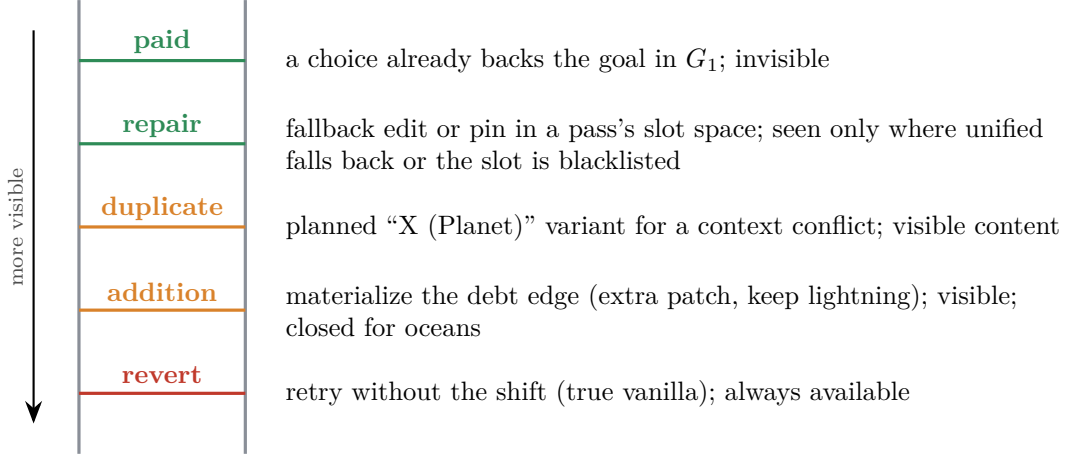


Figure 8: The settlement ladder. Each debt left after unified is settled by the highest rung that works. Theorem 5.6 covers the upper rungs. The last rung is always available: it falls back to the unshifted pipeline.

5.5 Comparison

	scaffolded	repaired	superposed
reference world	$G_1 + S$	G_1 , fallback $\sigma_{\text{ref}} \oplus \Delta$	$G_0 \cup G_1$
completable by	construction (PLAN-ETCHECK)	the repair search	Lemma 5.4, always
contradictions settled	before unified	before unified	after unified (ladder)
can use unified's choices	no	no	yes (repayment)
visible leftovers	all of S	only fallbacks used, blacklisted slots	only unpaid debts
needs new machinery	no	repair search	solvency mode, settlement

Table 2: The three references. The superposed reference is the only one where unified can fix contradictions itself. The repaired reference is the cheapest way to get most of the benefit with the current unified code.

Several shifts at once. Shifts compose: superpose all of them, take the union of their debt edges, and transport the goals through each shift in turn. Lemma 5.4 still applies, because every edge involved ends in an $\vee$ node. A new feature edge of one shift is a real edge in the superposition, so one shift can pay another's debts. Section 6 measures exactly that happening with oceans and resources together.

The glossary lists *debt* (a pending constraint with no known fallback bundle) as out of scope. Superposition is what brings it into scope. In $G_\square$ every pending constraint has a fallback bundle again, possibly one that goes through the old world, so a debt becomes something to *settle* rather than a hole in the proof.

6 Side investigation: fixing a small number of contradictions

Whichever reference is used, somebody has to find the repairs: the planetary stage for a repaired reference, unified's steering for a superposed one. This section asks how many contradictions a shift really creates, where they can be fixed, and whether the fixes lie in the space unified already searches. It combines theory with measurements on vanilla Space Age.

seed	ocean swap					resource swap					both				
	goals	causes	rep.	in	debt	goals	causes	rep.	in	debt	goals	causes	rep.	in	debt
1	151	4	10	3	4	24	3	5	3	3	164	8	12	4	6
2	151	4	10	3	4	139	3	5	4	3	76	8	13	6	6
3	151	4	10	3	4	24	3	10 [†]	10	3	166	8	11 [†]	7	7
4	151	4	10	3	4	51	3	5	4	3	164	8	13	6	6
5	151	4	10	3	4	51	4	12 [†]	12	4	166	9	15 [†]	9	8
6	151	4	10	3	4	139	2	3	3	2	76	7	11	5	5
7	151	4	10	3	4	139	4	12 [†]	12	4	166	9	14 [†]	9	8
8	151	4	10	3	4	139	4	8	6	4	76	9	15	7	7

Table 3: Contradictions of raw shifts, seeds 1–8. *goals*: failing goal pebbles (PLANETCHECK’s rules). *causes*: distinct lost feature pebbles on their vanilla witnesses. *rep.*: irreducible root-first repair set (ingredient slots). *in*: how many of those unified’s recipe-ingredients handler may randomize. *debt*: debt edges on the superposition’s solvency-first witness, i.e. what settling by addition would take. [†]: some goals are not reachable with *any* ingredient repair (Section 6.5); the repair set covers the others.

6.1 Setup of the measurements

The measurements ran in an isolated copy of the mod, which was stopped right after the planetary code had run (so no unified randomization), with the user’s current startup settings and seeds 1–8. For each seed there are three scenarios: the raw ocean swap, the raw resource swap, and both. “Raw” means the swaps are applied by the mod’s own `oceans.apply` and `resources.apply` with that seed’s assignment, but *without* scaffolds, recipe edits or extra patches. Each scenario records:

1. the contradictions D : `check.required_failures` of the shifted game against vanilla, i.e. exactly PLANETCHECK’s three rules;
2. the *causes* of each contradiction: the feature pebbles that its vanilla witness (`top.path`) uses and that the shifted world lacks;
3. staged repair sorts (Section 6.4) over ingredient slots, with repairs read off the witness and pruned with real sorts to an irreducible set;
4. the superposed graph $G_{\sqcup}$: whether it loses anything against the transported goals, and which debt edges its solvency-first witness needs.

The code is in Appendix A. Everything is logic-level: it answers what the logic graph says is reachable, which is the project’s source of truth.

6.2 Contradictions are counted in the wrong unit

Table 3 gives the first answer. A raw shift creates many failing goal pebbles, 24–166 per seed across the scenarios. These are not independent problems. They collapse to a handful of *causes*, the lost features themselves (2–9 per scenario), and to a similarly small number of *repairs*. One cause fails many goals at once: when Aquilo loses its ammoniacal ocean, cryogenic science is lost in every room where it was automatable, together with everything that needs it.

So “fixing a small number of contradictions” is the right framing, as long as contradictions are counted at the cause or repair level. Counted by goal pebbles, the planetary check’s own unit, the problem looks one to two orders of magnitude bigger than it is.

6.3 Repairs as relaxations

A repair changes what fills a slot. For the question *where* a repair is needed, the filler does not matter yet, which suggests treating an opened slot as filled by “anything available”.

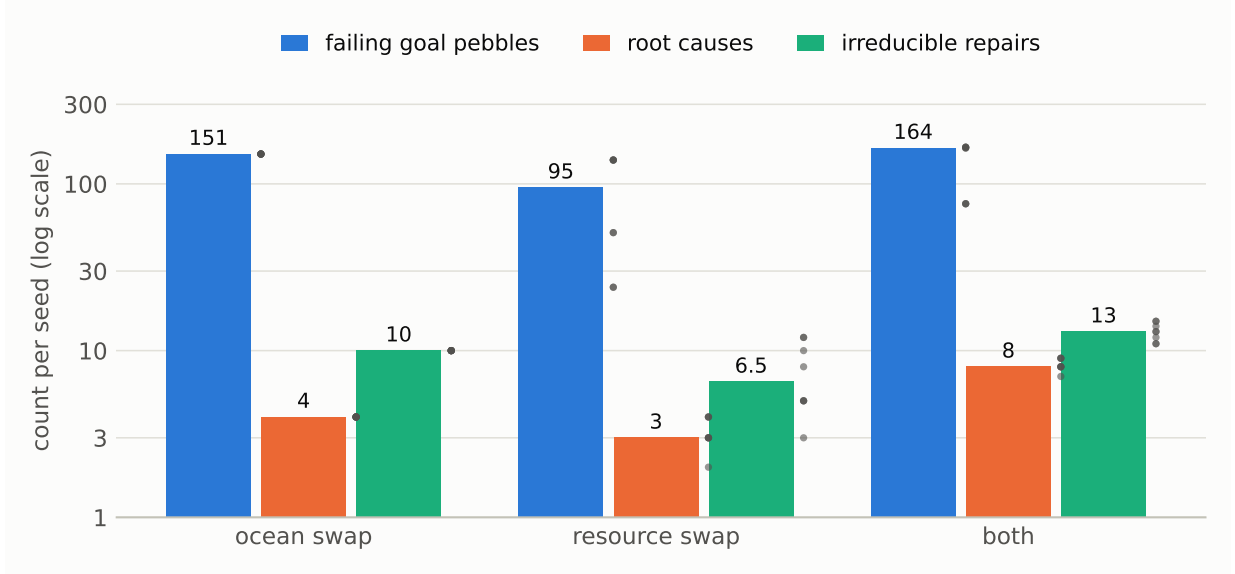


Figure 9: Goal failures, root causes and irreducible repairs per scenario (median over seeds 1–8, bars; per-seed values, dots; log scale). A raw ocean swap fails around 150 goal pebbles, which trace back to four lost oceans and are fixed by about ten slot repairs.

Definition 6.1 (Relaxed repair). For a set A of ingredient slots, $G \ominus A$ removes the in-edges of A (the recipe no longer needs those ingredients). A is a *relaxed repair* if $\Gamma_1 \subseteq \mathcal{R}(G_1 \ominus A)$.

Proposition 6.2 (Relaxation bound). *If a goal is not in $\mathcal{R}(G_1 \ominus K)$ for a class K of slots, then no way of refilling the slots in K reaches it.*

Proof. Any refilling G' is $G_1 \ominus K$ plus one in-edge per slot, and those edges end in $\wedge$ nodes (recipes). By Lemma 2.3, $\mathcal{R}(G') \subseteq \mathcal{R}(G_1 \ominus K)$. $\square$

A relaxed repair is monotone in A : opening more slots only reaches more. That makes it the right object for search. The *minimum* relaxed repair is still NP-hard, by reduction from set cover: give each set its own slot, which opens an $\wedge$ node that is otherwise unreachable, and make each element a goal $\vee$ node fed by the $\wedge$ nodes of the sets containing it. In practice the instances are tiny, so greedy pruning to an *irreducible* set (no single repair can be dropped) is enough.

Relaxation is also what separates the three kinds of fix:

- a **scaffold variant** is the relaxation made real. The original recipe stays and a copy with a different filler is added, so nothing that used the original loses anything (monotone, like $G \ominus A$);
- a **repair edit** realizes the relaxation by changing the recipe in place, which is only possible when one filler works in every context the recipe must keep (Section 6.6);
- a **leaf repair** opens slots far from the cause, in the consumers of what was lost.

6.4 Staged sorts: witnesses that repair only where needed

The earliest-provider witness of a random sort does not care whether it uses an opened slot or the original ingredient. On a fully relaxed graph it therefore patches every consumer it meets: a first version of this experiment, which removed every unified-randomizable slot at once, reported 44 “must-change” slots on 22 recipes for seed 1’s ocean swap. The fix is to make repairs *come later in the sort* than everything that needs no repair.

Definition 6.3 (Staged sort). Replace each relaxable slot $m \rightarrow r$ of class k by $m \rightarrow h \rightarrow r$ with an $\vee$ head h , and add an edge $gate_k \rightarrow h$, where $gate_k$ is an $\vee$ node with no inputs, so it

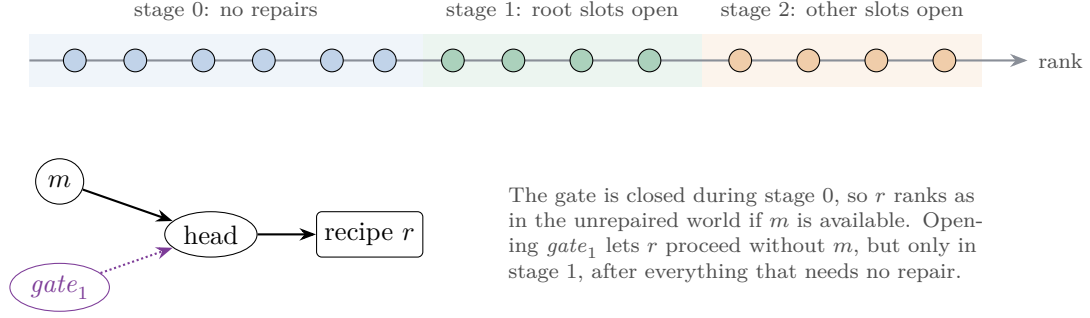


Figure 10: A staged sort. Relaxable slots sit behind closed gates that open one class at a time while the *same* sort continues, so repairs only enter witnesses where nothing earlier works.

starts closed. Sort. Then, for $k = 1, 2, \dots$, connect a source to $gate_k$ and *continue the same sort* (`top.sort`'s cached mode with one new connection).

Proposition 6.4 (Properties of a staged sort).

- (a) After stage k , the reached pebbles are exactly $\mathcal{R}(G_1 \ominus (\text{classes} \leq k))$;
- (b) every pebble first reached in stage s ranks after every pebble of earlier stages;
- (c) the earliest-provider witness of a goal first reached in stage s opens no gate of a later class. It goes through a class- s gate at a head only if the gate's pebble there ranks before the original ingredient's;
- (d) the gates on a witness form a relaxed repair.

Proof. (a) The continuation processes every consequence of the new connection until nothing new is open, so it reaches the least model of the extended program (Lemma 2.3). (b) New pebbles are appended. (c) Witness pebbles have lower ranks than the pebble they back, so by (b) they belong to stage s or earlier. (d) The witness is a derivation that uses only original edges and those gates (Lemma 2.2). $\square$

A staged sort ranks repair classes lexicographically: first see what needs no repair, then what needs only cheap repairs, and only then what needs expensive ones. With classes ordered by how undesirable a repair is, it is the practical form of a *minimal-repair witness* for this logic. It is a heuristic, not an optimum, because earliest-provider witnesses do not minimize the number of gates. The solvency-first witness of Section 5 is the same construction with the debt edges behind the gate. Figure 10 shows the mechanism.

Cost. In the measurements, with the machine under heavy load from other runs, the medians over the 24 runs were: about 4 s to rebuild the logic and sort it, 8 s for a three-stage staged sort, 4 s for the superposition's staged sort, and 46 s for pruning (one full sort per candidate repair; 35–111 s). Without pruning, a staged sort plus one verifying sort costs about three plain sorts, which is in line with the roughly four sorts today's scaffold selection takes. The group-halving pruning that `scaffolds.lua` already uses (`prune_groups`) would cut most of the rest.

6.5 Where the repairs are

Two orders of gates were measured:

root first $\rightarrow$ root slots (a lost material, in a recipe the losing planet made in vanilla: exactly what scaffold variants and resource edits change), then the remaining slots unified may randomize, then all others;

unified space $\rightarrow$ every slot unified's recipe-ingredients handler may randomize (its claim rules: not hidden, ingredient not in `blacklisted_pre`, recipe not in `blacklisted_dep`), then all others.

Tables 4 and 5 and Figure 11 give six findings.

	ocean swap	resource swap	both
goals fixable by root repairs	151	83 (1–139)	152 (76–164)
needing other unified slots too	0	0 (0–16)	0 (0–6)
needing any other slot	0	0	0
not fixable by any ingredient repair	0	0 (0–8)	0 (0–8)
irreducible repairs, root first	10	6.5 (3–12)	13 (11–15)
of which unified may randomize	3	5 (3–12)	6.5 (4–9)
goals fixable within unified’s move space	150	91 (16–139)	157 (75–163)
irreducible repairs, unified’s space first	26	12 (11–17)	30 (17–30)
of which outside unified’s space	1	0	1 (0–1)
debt edges (superposition witness)	4	3 (2–4)	6.5 (5–8)

Table 4: Where the repairs are: medians over seeds 1–8 with ranges in parentheses. Root repairs change a lost material’s slot in a recipe the losing planet made. “Unified’s move space” is every slot the recipe-ingredients handler may randomize.

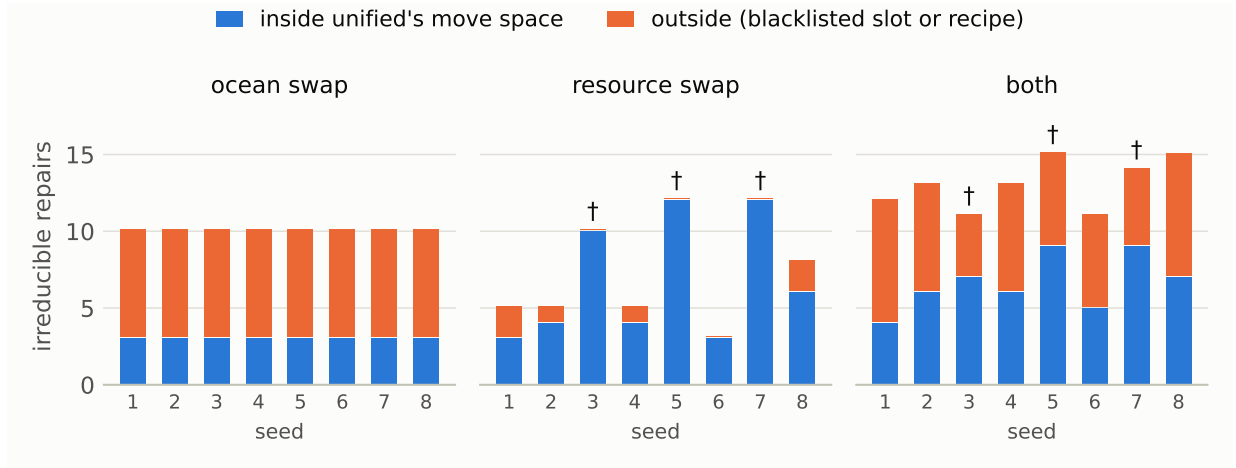


Figure 11: Irreducible repairs per seed (root-first order), split by whether unified’s recipe-ingredients handler may randomize the slot. The ocean swap’s repairs are the same on every seed, and 7 of its 10 are slots unified never touches. †: the run also has goals that no ingredient repair can bring back (Vulcanus lost calcite), so its repairs are the leaf repairs that fix the rest.

Root repairs almost always suffice. On every seed, every contradiction of the ocean swap comes back with root repairs alone. So does every contradiction of the resource swap, except when Vulcanus loses calcite. The root slots are the ones the planetary scaffolds and resource edits change, so the current planetary heuristics aim at the right place.

For oceans, most root repairs lie outside unified’s move space. The ten irreducible ocean repairs are the same on all eight seeds. On each of them, all four non-starting planets lost their ocean fluid; a water-for-water swap between Nauvis and Gleba is a valid assignment but did not come up. Seven of them are slots the recipe-ingredients handler never touches. Two are lava in `molten-iron-from-lava` and `molten-copper-from-lava`; four are water in `pentapod-egg`, `concrete`, `sulfuric-acid` and `heavy-oil-cracking` (water and lava are in `blacklisted_pre`); and one is ammoniacal solution in `ammoniacal-solution-separation`, a recipe in `blacklisted_dep`; all in `compat/vanilla.lua`. Only the three heavy-oil slots (`electrolyte`, `heavy-oil-cracking`, `lubricant`) are inside. For resources it is the other way round: most root repairs are slots unified already randomizes.

recipe	slot	seeds	kind	vanilla rooms	unified
ammoniacal-solution-separation	ammoniacal-solution	8	root	one	no
concrete	water	8	root	several	no
pentapod-egg	water	8	root	one	no
processing-unit	sulfuric-acid	8	root	several	yes
electrolyte	heavy-oil	6	root	several	yes
heavy-oil-cracking	heavy-oil	6	root	several	yes
heavy-oil-cracking	water	6	root	several	no
lubricant	heavy-oil	6	root	several	yes
molten-copper-from-lava	lava	5	root	one	no
molten-iron-from-lava	lava	5	root	one	no
acid-neutralisation	sulfuric-acid	4	root	one	yes
plastic-bar	coal	4	root	several	no
tungsten-carbide	tungsten-ore	4	root	several	yes
tungsten-plate	tungsten-ore	4	root	several	no
rocket-part	low-density-structure	3	leaf	several	yes
rocket-part	processing-unit	3	leaf	several	yes
space-platform-starter-pack	processing-unit	3	leaf	several	yes
space-platform-starter-pack	space-platform-foundation	3	leaf	several	yes
space-platform-starter-pack	steel-plate	3	leaf	several	yes
advanced-oil-processing	crude-oil	2	root	several	yes
advanced-oil-processing	water	2	root	several	no
carbon	coal	2	root	several	yes
light-oil-cracking	water	1	root	several	no

Table 5: Irreducible repairs for the combined shift (oceans and resources, root slots ordered first), with the number of seeds (of 8) each appears in. *kind*: a root slot, or a leaf slot that the calcite seeds need on top. *vanilla rooms*: “one” marks recipes that today’s planetary rule edits in place. *unified*: whether unified’s recipe-ingredients handler may randomize the slot.

Inside unified’s move space, repairs become leaf repairs. With unified’s slots ordered first, every contradiction but one comes back. The exception is the pentapod egg’s water slot, for which no other route exists. The irreducible sets, however, are larger and farther from the cause: over the 8 seeds measured this way, 26 slots instead of 10 for the ocean swap, 11–17 instead of 3–12 for the resource swap, and 17–30 instead of 11–15 for the combined shift. On the ocean swap that is 2.6 times as many on every seed, and 2–2.7 times for the combined shift on the seeds where Vulcanus keeps its calcite. On the calcite seeds (3, 5, 7) the gap nearly closes, because the root-first order needs leaf repairs there too. About half of these repairs are in recipes other planets make too (rocket parts, rocket silos, starter packs, landing pads, supercapacitors). That share is no worse than for root repairs, many of which are also in shared recipes such as **concrete** and **processing-unit**. The difference is the count and the distance from the cause: the goal recipes and their consumers get patched one by one, instead of the one recipe that fed the lost material to the chunk. Unified could fix most contradictions within its current rules, but it would need several times as many changes, spread over the consumers.

Some contradictions are structural. When Vulcanus loses calcite (seeds 3, 5 and 7), eight goal pebbles come back with no ingredient repair at all, which Proposition 6.2 establishes from one staged sort: the foundry, big mining drill, acid neutralisation and metallurgic science pack recipes in Vulcanus’s isolatable contexts. The obstacle is a technology chain, not an ingredient. **calcite-processing** is researched by mining calcite, and **foundry** lists it as a prerequisite (`data/space-age/prototypes/technology.lua:629-654, :774`). A technology that is isolatable on Vulcanus therefore needs calcite minable on Vulcanus. Only an extra calcite patch (the addition rung, which the resource stage already implements as “repairs”) or a changed trigger can fix these.

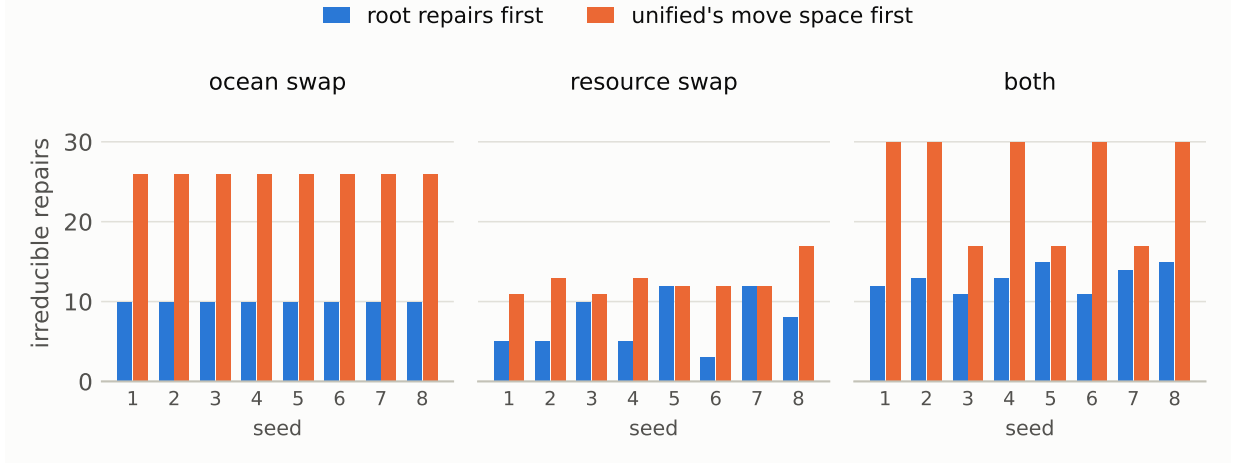


Figure 12: Irreducible repairs with root slots ordered first (blue) versus with unified’s whole move space ordered first (orange), per seed. For the ocean swap, unified’s move space needs 26 slot changes instead of 10 on every seed, because it can only reach the contradictions from the consumers’ side. The gap nearly closes on seeds 3, 5 and 7, where Vulcanus loses calcite and the root-first order needs leaf repairs as well.

Shifts interact. Contradictions are not additive. On seeds 2, 6 and 8 the combined shift fails 76 goal pebbles, half as many as the ocean swap alone (151). On seed 2, 90 of the ocean swap’s failures come back once the resource swap also runs; they are science-pack sets and items that were reachable nowhere. Fifteen new ones appear, all in Vulcanus’s rocket building. One shift can bring together what another separated. A superposed reference handles this for free, because every new feature edge of every shift is real in it.

Superposition always works, and its debt is small. The superposed graph lost nothing in any of the 24 runs, and every edge that exists only in the unshifted graph ended in an $\vee$ node, as Table 1 predicts. Its solvency-first witness needs 4 debt edges for the ocean swap (one per planet whose ocean changed), 2–4 for resources and 5–8 for both. Settling everything by addition would take one old feature back per lost feature at most, which bounds how much the addition rung can ever cost.

6.6 Context conflicts: when a repair must be a duplicate

A relaxed repair says where to change a recipe, not what to put there. A recipe has a single ingredient list, and it must work in every context the recipe keeps.

Definition 6.5 (Context conflict). For a slot of recipe r with required contexts C_r , let $\mathcal{A}(c)$ be the set of materials established in context c before $\langle r, c \rangle$. The slot is in *conflict* if $\bigcap_{c \in C_r} \mathcal{A}(c) = \emptyset$.

Proposition 6.6. *A conflicting slot cannot be repaired in place. Splitting r into copies, one per group of contexts that share a filler, repairs it. A recipe whose vanilla contexts all lie in one room can conflict only between the ability variants of that room.*

This is the precise version of the rule the planetary stage already follows: “originals only ever used on that planet in vanilla are edited in place”, while the others keep a planned “X (Planet)” duplicate. The rule is conservative. A *shared* recipe (made on several planets in vanilla) does not always conflict, since the new filler may be available wherever the recipe is required. That happens, for instance, when the other rooms only require the recipe without isolatability and can import the filler. So duplicates could be rarer than they are now. Figure 13 illustrates the definition. In the measurements, 6 of the 10 ocean repairs and 74 of the 104 combined-shift

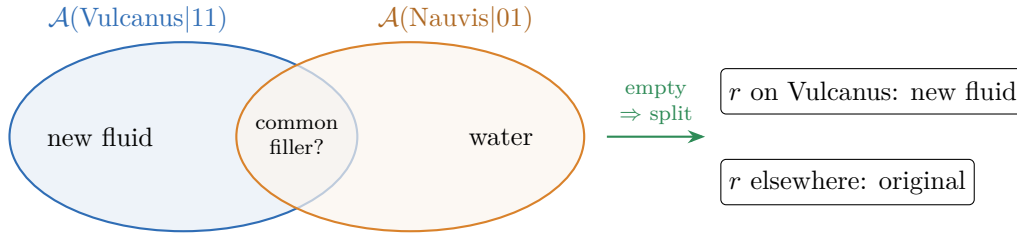


Figure 13: A context conflict. If no single filler is available before r in every context r must keep, an in-place edit breaks some context. Splitting r per group of contexts (a planned duplicate) does not.

repairs (over all seeds) are in recipes made in more than one room in vanilla. Today’s rule would duplicate all of them. With the context-conflict test, those whose new filler is available wherever the recipe must stay can be edited in place instead.

6.7 What this means for “unified fixes the contradictions”

1. **The contradictions of a planetary shift are few and local:** a handful of causes and about a dozen repairs, mostly at the root. That supports integrating shifts through a repaired or superposed reference instead of scaffolding the whole world first.
2. **Unified cannot make the natural repairs today.** Its move space excludes most ocean root slots, and the repairs it can reach are leaf repairs: more of them, and far from the cause. So “unified fixes the contradictions” needs either *repair-only moves* (blacklisted slots allowed for repairs, as pins, never for random choices) or a repaired reference computed before unified, which unified then re-randomizes freely wherever it can.
3. **Structural contradictions need other rungs of the ladder:** the addition rung for resources (already implemented), technology-trigger moves (a different handler), or a revert. The relaxation bound identifies them cheaply, so the ladder can jump straight to the right rung.
4. **The superposition is always a valid starting point.** It needs no search, and its debt is at most one old feature per moved feature. Debt-aware promotion therefore has a bounded worst case: settling everything by addition, which the user already accepts for resources (“a couple extra resources on a planet”).

7 Putting it together

7.1 A pipeline with shifts inside the relay

```
1 G0  <- logic of vanilla; home sets fixed from G0
2 tau <- shift assignment (oceans, resources, climate)
3 G1  <- G0 with tau applied raw (no scaffolds, edits or patches)
4 Gam1 <- transported goals: rules 1-3 of planetary/check.lua
5 D    <- Gam1 minus Reach(G1)                                (one sort)
6 if D is not empty:
7     A    <- staged sort over root, unified, other slots;
              gates on the goals' witness; prune                (Section 6)
8     Delta <- a filler for each slot in A: the planetary
              substitution first (new fluid, replacement resource),
              in place unless the slot is in context conflict,
              else a planned "X (Planet)" duplicate
9     goals still failing -> superpose G0 and G1, carry the debt edges
10 reference <- (G1 with fallback sigma_ref + Delta, or the superposition)
11 unified: first pass, generic handlers, promotion against the reference;
              if superposed: establish_solvent, no-new-insolvency rule,
              candidates steered by the repair plan
12 settle what the solvency-first witness still owes:
              pin / edit -> duplicate -> addition -> retry without the shift
13 check the final game against Gam1 computed from TRUE vanilla
              (planetary rules) plus the promised pebbles (MECHCHECK's model check)
```

Steps 1–8 and 10–11 need no new theory. They are today’s planetary stage with a repair search in place of the scaffold selection, followed by today’s unified. Step 9 and the debt-aware part of step 11 are the new machinery (Section 5). Step 12 generalizes what the planetary stages already do on failure (drop edits, add patches, revert a climate, undo a stage). Step 13 changes which goals the last check uses: once unified can change what the planetary stage would have fixed, the post-planetary world can no longer serve as “the original”, and the check has to compare against true vanilla under the transported goals.

Today a failed check no longer stops the game from loading. A failed UNIFIEDCHECK retries the attempt and keeps the last one. A failed final MECHCHECK sets `check_ok=false` in the `propertyrandomizer-reachability-data` mod-data, which the randomizer panel shows (`data-final-fixes.lua`, `scripts/gui.lua`). The ladder is meant to make that path rare. Its last rung, retrying without the shift, falls back to the unshifted pipeline, so turning a shift on never makes a seed worse than it would be without it.

7.2 Context-shifting first pass

The same framework covers the other context shift the user has in mind, a first pass that moves travelers to slots whose contexts differ from their own (e.g. lava on Gleba, or a Vulcanus-only material in a Nauvis slot). Monotone matching admits a move only if the slot supplies every needed context earlier. That is precisely a *zero-debt* shift. Allowing a non-monotone move means:

1. superpose the trav’s old and new connections (both are $\vee$ in-edges of the slot’s item in first pass’s split graph, so Lemma 5.4 applies);
2. transport the goals that follow the trav, if any are feature-like;
3. run the staged sort to get a repair plan for the debt, and accept the move only if the plan exists within the later passes’ move spaces (Proposition 6.2 rules out moves that no refilling can fix);
4. settle at the end as in Section 5.

Random non-monotone moves failed badly in the earlier tempering experiment. In this framework such moves become bounded-debt moves with an explicit repayment plan. The budget, meaning how much debt a first pass may take on, is a new tuning knob.

7.3 Recommendations, cheapest first

1. **Repaired reference from the planetary stage (no change to unified).** Replace the scaffold and patch selection with a staged sort over repair classes, pruned with real sorts, and realize repairs in place unless the slot is in context conflict. That keeps today’s order (planetary, then unified) and should turn many planned duplicates into fallback edits. The context-conflict test replaces the “only used on that planet” rule, which is stricter than necessary.
2. **A shared staged-sort helper.** The gate construction (Proposition 6.4) is small. It belongs in one helper next to `top.sort`, used by scaffolds, resource repairs and settlement, with a plain-Lua test on a toy graph for properties (a)–(d). This follows the “enforce concepts in code” rule.
3. **Repair-only moves in unified.** Let recipe-ingredients accept a candidate for a slot on a repair plan even when the slot is blacklisted (water, lava), but never as a random choice. With (1) in place, that means keeping the repaired filler as a pin, which promotion already supports for recycling.
4. **Superposed reference and debt-aware promotion**, behind a flag. Start with resources, whose addition rung (extra patches) already exists, then climate. The additions to promotion are an `establish_solvent` mode, the local no-new-insolvency rule, and a settlement pass after unified.
5. **A reference object.** Pass one object (graph, fallback assignment, sort, goals) between passes instead of letting each stage rebuild “vanilla” from `old_data_raw`. The relay of Section 3 becomes explicit, and a shift becomes the one kind of pass that is allowed to change the goals.

7.4 Decisions that are the user’s

These are design choices about what the game should be, not facts the code or data can settle:

- Q1. **Which goals the final check uses when a shift is on.** Today MECHCHECK compares against the post-planetary world. In an integrated pipeline it would have to be the transported goals Γ_1 computed from true vanilla (the planetary rules). Is that the intended spec?
- Q2. **Blacklisted slots as repairs.** Water and lava are excluded from unified’s ingredient randomization (`compat/vanilla.lua`). Most ocean root repairs are exactly those slots. May a *repair* (fallback or pin) use them, while random choices still may not?
- Q3. **In-place edits of shared recipes.** When a shared recipe has no context conflict, should it be edited in place (every planet sees the new filler) or still get a planet duplicate (today’s rule)?
- Q4. **The addition rung for oceans.** It is closed by the “oceans move as tiles, fluid changes with them” decision. Should it stay closed even as a last resort before reverting?
- Q5. **What “revert” means.** Retry the whole attempt without the shift (always sound), or revert per feature group (cheaper, but only sound if nothing unified promised uses the new feature edges)?

A Experiment details

Where it ran. An rsync'd copy of the mod (live working tree of 26 September 2026, including uncommitted changes, without `.git` and the 2.4GB cost log), with its own Factorio config and write-data directory, the user's current `mod-settings.dat` with only `propertyrandomizer-seed` changed, the Space Age mod list (`tests/mod-configs/sa.json`), and Factorio 2.1.20 headless. `data-final-fixes.lua` in the copy calls the measurement module right after `randomizations/prefixes` and then stops with an error. Nothing else of the randomizer runs, and the live repository was not modified.

Scenarios. The ocean assignment is `oceans.random_assignment("planetary-oceans")` and the resource assignment `resources.random_assignment(resources.slots(), "planetary-resources")`, the same RNG keys the planetary stage uses. They are applied with `oceans.apply` and `resources.apply` to a fresh copy of `data.raw`, with no scaffolds, recipe edits or extra patches. The logic is rebuilt with `logic.build(true)`, and all sorts use complex contexts and home contexts, with home sets fixed from the vanilla sort, like PLANETCHECK.

Measures.

- *Contradictions:* `planetary_check.required_failures(before, after, {})`, with `moved_features` set for the scenario.
- *Causes:* for each contradiction, the pebbles of its `top.path` witness in the vanilla sort whose node is a feature node (`planetary_feature="oceans"`, or an entity node of a resource prototype) and which the shifted sort lacks.
- *Slot classes:* ingredient edges into recipe nodes (from `item` or `fluid-temperature-range` nodes). *Root:* the material is one the recipe's room lost (ocean fluid or mined product of a lost resource), and the recipe had a vanilla pebble in that room. *Unified:* `recipe-ingredients'` claim rules minus its cost check (recipe not hidden, material not `blacklisted_pre`, recipe not `blacklisted_dep`), which over-approximates its move space slightly.
- *Staged sorts:* gates as in Section 6.4, continued with `top.sort`'s cached mode. A slot counts as repaired on the witness when the witness's head pebble is backed by the gate, i.e. the gate's pebble in that context ranks before the ingredient's. The first version of the experiment compared against the head's own rank instead, which missed gates that ranked before a later-reached ingredient. Pruning then failed on some seeds, and the fixed version was rerun for all seeds.
- *Pruning:* with only the witness's slots opened (one gate, open from the start), drop slots one at a time (the costliest class first), keeping a drop whenever every contradiction is still reached in a full sort. The result is irreducible, not necessarily minimum.
- *Superposition:* the shifted graph plus every vanilla-only edge behind a gate (an $\wedge$ node with the edge's start and the gate), sorted closed, then continued with the gate open. Debt edges are the gated $\wedge$ nodes on the witness of all contradictions.

Caveats.

- Relaxations are optimistic about fillers: they locate repairs without choosing what fills a slot. The planetary substitution (the new fluid or replacement resource) is one concrete filler. Earlier planetary work found it sufficient in practice: across 8 seeds no conversion recipe was ever needed.
- Causes come from one witness per contradiction, so a contradiction with two vanilla routes is attributed to the causes on the earliest one only.
- Everything is logic-level. In-game effects the logic does not model still apply, for example furnace-type machines choosing their recipe by input (`FurnacePrototype`, `doc-html/`

`prototype-api.json`), which is why resource edits skip furnace categories whose recipes already take the new ingredient.

Files. Everything is in `notes/context-shift-report/experiment/`. The measurement modules are `experiment-contradictions.lua.txt` and `experiment-ospace.lua.txt`, stored as `.lua.txt` so the repository's Lua hooks ignore them. `make-copy.sh` builds the scratch mod copies, `run.py` runs seeds, and `parse.py`, `tables.py` and `charts.py` produce this report's tables and charts from the per-seed logs in `out-v3/` and `out-u/`.

B New terms

Reference

A completable world with a fallback assignment, a sort, and goals it reaches (Definition 3.3). Promotion needs one, and vanilla is only the first.

Relay

Multipass as a sequence of passes, each receiving a reference and handing one on.

Feature edge

An edge into a node that says a planetary feature is present or usable (from a room-level node, or from a tile's pump node for oceans). All current ones end in $\vee$ nodes.

Shift

A pass that replaces feature edges. It is the one kind of pass that changes the goals.

Transport

How goals follow a shift: feature goals move (or are dropped, as now), identity goals stay (with relaxed isolatability), recipe goals stay.

Contradiction

A transported goal that the shifted world does not reach.

Cause

A lost feature pebble on a contradiction's vanilla witness.

Chunk

The part of a witness that a lost feature pebble dominates, which has no root after the shift.

Root / leaf repair

A fix at the chunk's lowest consumers, or in the consumers above the chunk.

Repair

A change of the *fallback* assignment within a later pass's slot space. It is not content.

Relaxed repair

A set of slots whose removal (filled by anything) reaches the goals. It locates repairs.

Staged sort

A sort continued as gates open, one repair class at a time, so witnesses use repairs only where needed.

Superposition

The union of the old and new worlds. It is always a reference when feature edges end in $\vee$ nodes.

Debt edge

A feature edge of the old world kept in the superposition. It must be paid or settled.

Solvent

Backed without debt edges.

Settlement ladder

Paid, repair, duplicate, addition, revert: the order in which leftover debts are settled.

Context conflict

No single filler works in every context a recipe must keep, so an in-place repair is impossible and a duplicate is needed.